PHOT 110: Introduction to programming

Lecture notes

Michaël Barbier

2026-04-15

Table of contents

Preface

This is a Quarto book.

To learn more about Quarto books visit https://quarto.org/docs/books.

1 Introduction

These are the lecture notes of the introduction to programming course given to 1th year photonics students at IZTECH. The course book being used is Hans Petter Langtangen (5th edition) See Knuth (1984) for additional discussion of literate programming.

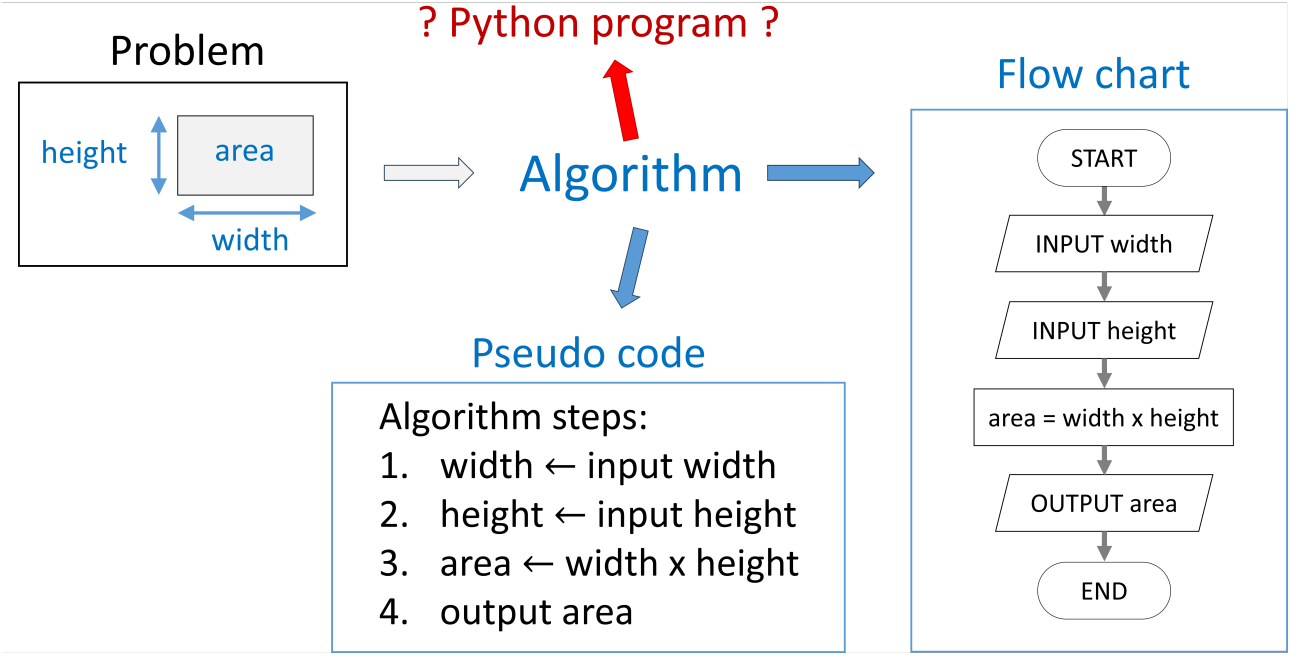
Hans Petter Langtangen https://hplgit.github.io/

https://hplgit.github.io/scipro-primer/

https://hplgit.github.io/scipro-primer/slides/index.html

Book on Numerical Methods

Computer science is the discipline which studies solving problems by designing and implementing algorithms executed on a computer. An algorithm is here a recipe or procedure containing simple steps to solve the problem and a stopping criterium which determines when the algorithm is finished and the solution found. When an algorithm is implemented on a computer in a computer language or programming language then we call the procedure a program. Here, we will provide the basics to solve scientific problems and implement numerical methods in **Python**. A large part is dedicated to describing the Python syntax, i.e. the grammar of this computer language.



2 Python basics

Python is an interpreted language, this means that the Python script is not first compiled to machine-code (an executable), but there is an interpreter (also a computer program) that executes the Python commands for us. This saves us the time and extra step to compile the script, but we need to have the Python interpreter installed and programs typically run slower. Later on, we will see that the last performance issue often can be mitigated by using highly optimized (and compiled) helper libraries for computationally heavy tasks, e.g. the Numpy and Scipy packages.

2.1 Python program structure

A Python script is a sequence of **statements** (algorithm steps), and **definitions** (functions, classes, …)

- Definitions are evaluated, i.e. they can be used afterwards
- Statements are executed

In Python we can execute a program in two ways:

- Execute statements **one by one** in the **Python Console**, this is directly talking to the Python interpreter, or
- Statements **stored** as a sequence in a **script file**

2.2 The Python console

Execute statements one by one in the Python Console

```
>>> print(6)
6
>>> a = 10
>>>
```

Useful for:

- quick calculations
- testing the working of commands

2.3 Python scripts

Statements stored in a script file

```
a = 10
b = 25
prod = a * b
print(f"{a} times {b} = {prod}")
```

```
10 times 25 = 250
```

Useful for:

- running a program multiple times
- incorrect lines can easily be found and corrected
- larger programs

2.4 Data objects

In Python **everything is a data object**: numbers, text, functions, class instances, etc. Python **statements** perform actions on data objects.

Data objects can be scalar, of a basic data type, or composite (combining some of the basic data types). Some scalar data objects are listed in the table below:

Type	Description	(Example) values
`bool`	Boolean value	`True`, `False`
`int`	Integer numbers	.. , `-2`, `-1`, `0`, `-1`, `-2`, ..
`float`	Floating point	`3.56`, `23e-3`, `0.0079`
`NoneType`	Indicates no object	`None`

2.4.1 Object type

Every object has a type, for example a decimal number has as type `float`:

```
print(type(6.22e23))
```

```
<class 'float'>
```

It is possible to convert between certain types, this we call **type casting**. For example, a logical true (`True`) corresponds to an integer with value 1. While the logical false (`False`) would be casted into `0`.

```
it_is_raining = True
print(type(it_is_raining))
an_integer = int(it_is_raining)
print(type(an_integer))
```

```
<class 'bool'>
<class 'int'>
```

When doing specific operations between objects, objects can undergo automatic type casting. For example, when multiplying an integer with a decimal number, the product could become a decimal number, therefore the Python interpreter gives us the result as a decimal number (a `float`). When Python prints a decimal number it shows the decimal point, so we can understand what the result has as type. Some typical cases of automatic type casting are:

`float` <= `int * float`
`float` <= `int + float`

`float <= int / int`

We can see this in practice in following code:

```
a = 4
b = 0.357
print( f"Type of (a * b) = {type(a * b)}" )
print( f"Type of (a / b) = {type(a / b)}" )
print( f"Type of (a + b) = {type(a + b)}" )
print( f"Type of (a / 25) = {type(a / 25)}" )
```

```
Type of (a * b) = <class 'float'>
Type of (a / b) = <class 'float'>
Type of (a + b) = <class 'float'>
Type of (a / 25) = <class 'float'>
```

Here, the Python function `type(a)` gives the type of object with name `a`, and `type(a * b)` the type of the product `a` times `b`, etc.

2.4.2 Variables

To be able to reuse values and to operate on them we use variables, that is, we assign a name to them. A **variable** is a name bound to an **object**:

- The object is **assigned** to the variable,
- In pseudo code indicated by ←

In python the assignment operator is the "=" sign:

```
a_variable_name = 4.5
```

The variable can be re-assigned another value or object:

```
v = 4
v = v + 2                          # v becomes 6
v = "Now v is assigned text"       # v has type str
```

The assignment operator is thus not the same as the "is equal to" operator from mathematics, this is especially clear when considering the statement `v = v + 2` above.

2.5 Expressions & Operators

Objects can be combined by **operators** in **expressions**. Typically we consider arithmetic operators and logical operators working on our basic object types (numbers and logical values), although we will see more complex operators later on.

Most used arithmetic operators:

symbol	description	example
**	Power	3**2 = 9
/	Division	5 / 4 = 1.25
*	Multiplication	3 * 4 = 12
+	Addition	5 + 7 = 12
-	Subtraction	6 - 9 = -3
%	modulo	34 % 6 = 4

Most used logic operators:

symbol	description	example
`<`, `>`	smaller/larger than	`5 > 4` → `True`
`==`	is equal to	`3 == 6` → `False`
`<=`, `>=`	smaller/larger or equal	`5 <= 5` → `True`
`and`	boolean AND	`True and False` → `False`
`or`	boolean OR	`True or False` → `True`
`not`	boolean NOT	`not True` → `False`

2.5.1 Structure of an expression

An expression can be built using following rules recursively:

- `<expression>` $\overset{def}{=}$ `<object>` (an object is an expression)
- `<expression>` $\overset{def}{=}$ `<operator> <expression>`
- `<expression>` $\overset{def}{=}$ `<expression> <operator> <expression>`

Expressions result into values and can be assigned to variables:

```
x = 4
y = 2*x**2 + 3*x - 5
print(f"The value of y = {y}")
```

```
The value of y = 39
```

2.5.2 Operator precedence and brackets

Precedence of operators is similar to mathematics: taking a power has priority over multiplication(division) and multiplication(division) has priority over addition(subtraction). For equal priority we apply the operators from left to right.
List of precedence of operators can be found on the python.org website: https://docs.python.org/3/reference/expres
precedence

Round brackets can be used to give priority or add clarity:

```
x = 2
y = (12 - x) / 10
print(f"The value of y = {y}")
```

```
The value of y = 1.0
```

2.6 Text objects

Text objects are called strings: the type is `str` A string is defined by using single or double quotes:

```
"This is a string, single 'quotes' can be used here"
'Here we can use double "quotes" inside it'
```

Strings can also cover multiple lines, for that we use triple quotes:

```
""" This string runs over multiple lines.
Another line starts here.
And another one.
"""
```

Also triple single quotes work as well.

2.6.1 Operators working on text objects

Appending one string to another with the "+" operator:

```
a_verb = "flies"
print("The balloon" + a_verb)
print("The balloon" + " " + a_verb)
```

```
The balloonflies
The balloon flies
```

Multiplying strings:

```
print(5 * "balloon ")
```

```
balloon balloon balloon balloon balloon
```

Casting a string to a number and vice versa:

```
print(5 * int("100"))
print(10 * str(50))
```

```
500
50505050505050505050
```

2.6.2 Formatting strings and numbers

A formatted string is a string with prefix `f` or `F`:

```
the_radius = 25
formatted_str = f"A circle with radius {the_radius} m"
print(formatted_str)
```

```
A circle with radius 25 m
```

More complex formatting:

```
a = 1/6; b = 0.0145; c = 23e-6
print(f"The product {a} x {b} = {a * b}")
# Use format {variable:No_space.No_sign_digits}
print(f"{a:8.2} x {b:8.2} = {a * b:8.2}")
print(f"{a:8.2} x {c:8.2} = {a * c:8.2}")
```

```
The product 0.16666666666666666 x 0.0145 = 0.002416666666666667
    0.17 x    0.015 =   0.0024
    0.17 x  2.3e-05 =  3.8e-06
```

2.7 Python reserved words or keywords

- A list of words **reserved** for Python
- You cannot name variables (or functions/classes) similar

```
# List of keywords can be found in the keyword package
import keyword

print(keyword.kw_list)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue
```

2.8 input & output

When not in interactive mode:

- `input()` can be used to ask user text input
- `print()` can be used to output text

```
# parameters
pizza_price = 200
extra_cheese_price = 20

# input accepts a string parameter
n_pizza = input("How many pizza's do you want: ")
extra_cheese = input("""Do you want extra cheese?\n
    For yes press [1]\n
```

```
    For no press [0]\n
Enter your choice""")

# Output to the user
print("The total cost is {int(n_pizza * (pizza_price + extra_cheese)}")
```

2.9 Indentation

- Is the amount of space preceding statements
- Indentation should be the same within a code block
- **Spaces** and **Tabs** are different
- Following code will give an `IndentationError: unexpected indent`

```
a = 4
b = 2
 c = 3
y = a * (b + c)
```

```
IndentationError: unexpected indent (3759259151.py, line 3)
  Cell In[33], line 3
    c = 3
    ^
IndentationError: unexpected indent
```

2.10 Import statements

Python packages can be imported in multiple ways

```
# import the package with its name
import math

area = 3**2 * math.pi
print(f"Area of a circle with radius 3 = {area}")

# import functions of the package separately
from math import sin, cos, pi, sqrt

angle = 23/180*pi
formula = sqrt(2/3) * sin(angle) + cos(angle)

# import packages or functions and rename them
from math import sqrt as sr

print(f"The square root of 2 = {sr(2)}")
```

2.11 Tidy and Documented code

A tidy writing style has many benefits beyond just being neat. We can defined it by adhering to following points:

- Writing **readable** code
- Clear and not too long variable names
- Use spaces in a consistent manner
- Use a Python code formatter such as **Black**: https://github.com/psf/black
- Document code both with **comments** and **docstrings**

2.11.1 Comment lines

- Text of a line after a hash symbol is ignored
- The hash symbol can be in the beginning of the line:

```
# A comment on a single line
```

Comments can explain the next line of code

```
# Convert Matlab-indices to zero-based
ind = m_index - 1
```

Or the comment can start after a statement

```
y = 12 + x  # An inline comment: a hash symbol after 2 spaces
```

2.11.2 Multi-line comments

Multi-line strings can be used as comments, but

- they are not meant as comments (not ignored)
- they can become docstrings (documentation-strings)

```
'''
This multi-line string is not assigned to a
variable, therefore doesn't influence your code
directly
'''
```

2.11.3 Docstrings

- Docstrings are multi-line strings providing documentation

- They populate the `__doc__` variable
- The help() function uses the `__doc__` variable

```
import math
print(math.__doc__)
```

```
This module provides access to the mathematical functions
defined by the C standard.
```

```
import math
help(math.sin)
```

```
Help on built-in function sin in module math:

sin(x, /)
    Return the sine of x (measured in radians).
```

- We will revisit Docstrings for functions and classes later.

2.12 Program Errors

Different types of errors can be encountered

- **Syntax errors**: code inconsistent with the Python language, the code will not run (i.e. not start).
- **Runtime errors**: exceptions occurring while the program runs, the code will crash.
- **Semantic errors**: the code does not what was intended. This type of errors is the hardest to track.

2.12.1 Syntax error examples

- Usage of unknown identifiers (variable, function, class)

```
positive_number = uint(45)
```

```
NameError: name 'uint' is not defined
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[36], line 1
----> 1 positive_number = uint(45)
NameError: name 'uint' is not defined
```

- Misaligned indentation

```
a = 34
 b = 5
print(f"The sum is {a+b}")
```

```
IndentationError: unexpected indent (3752930843.py, line 2)
  Cell In[37], line 2
    b = 5
    ^
IndentationError: unexpected indent
```

- Mistaken symbol ":" instead of ";"

```
a = 3: b = 5
```

```
SyntaxError: invalid syntax (173182802.py, line 1)
  Cell In[38], line 1
    a = 3: b = 5
         ^
SyntaxError: invalid syntax
```

2.12.2 Runtime error examples

- Division by zero

```
fraction = 1 / 0
```

```
ZeroDivisionError: division by zero
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[39], line 1
----> 1 fraction = 1 / 0
ZeroDivisionError: division by zero
```

- Type mismatch

```
netto_price = 230.50
kdv_ratio = 0.21
print("Price: " + netto_price + " + " + kdv_ratio + " kdv")
```

```
TypeError: can only concatenate str (not "float") to str
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[40], line 3
      1 netto_price = 230.50
      2 kdv_ratio = 0.21
----> 3 print("Price: " + netto_price + " + " + kdv_ratio + " kdv")
TypeError: can only concatenate str (not "float") to str
```

Knuth, Donald E. 1984. "Literate Programming." *Comput. J.* 27 (2): 97–111. https://doi.org/10.1093/comjnl/27.2.97.